PM101 · SESSION 4

Prioritization Workshop

Saying no to 80% of good ideas — and defending it out loud.

> **You will not leave this room with a framework you memorized. You will leave having used one, broken it on purpose, and defended the call to someone who disagreed with you.**

Format: 2 short activities + 1 long, hands-on activity in a real product management tool (Jira or Azure DevOps). Running example throughout: **Swiggy Instamart.**

OPENING SCENARIO · POLL

You can ship exactly one. Choose.

It's the last sprint of the month at Swiggy Instamart. Three things are on the table and only one can ship:

A · CEO mandate

A "10-minute delivery guarantee" banner across the app — a brand promise leadership wants live before a competitor's ad campaign lands.

B · Partner threat

Your largest dark-store partner is threatening to walk over a recurring **inventory-mismatch bug** — items shown as in-stock that aren't.

C · Quiet win

Engineering says a **smart reorder suggestion** feature for dark-store restocking is one sprint from done — no one is asking for it loudly, but it could quietly cut waste.

Which one do you ship this sprint?

- **A — Ship the 10-minute guarantee banner**
- **B — Fix the inventory-mismatch bug**
- **C — Ship smart reorder suggestions**

0 responses so far

- **Your response only (live sync unavailable)**

Hold onto your answer. You'll be asked this exact question again at the end of the session.

FRAMEWORK · RECAP

Win-Win vs Trade-Off

Every prioritization call is one of four mindsets. Most PMs default to the wrong one out of habit.

Decision Type	Mindset	When to Use	PM Example
Win-Win (Synergistic)	"How can both outcomes win?"	Early discovery, when goals seem conflicting but might be aligned	Build onboarding that improves activation *and* retention.
Trade-Off (Selective)	"What matters most now?"	Execution phase, when time/resources are fixed	Ship MVP with core features, delay personalization.
Win-Win (Strategic)	"Reframe the problem to create value for multiple sides."	When teams or stakeholders have conflicting priorities	Internal automation that reduces ops load *and* improves accuracy.
Trade-Off (Tactical)	"Choose the higher impact with available capacity."	When short-term focus is needed	Prioritize iOS app polish over Android parity due to market share.

INTERACTIVE · REFRAME CHECK

Is Slide 2 actually a disguised Win-Win?

Look again at the Instamart scenario. Most people call it a Trade-Off (Tactical) — pick the highest-impact option and move on. But is there a reframe hiding in plain sight?

Is there a Win-Win (Strategic) reframe here?

Yes — B is actually a precondition for A to be credible

No — this is a genuine either/or Trade-off

0 responses so far

- **Your response only (live sync unavailable)**

Hint: does fixing the inventory-mismatch bug (B) also make the 10-minute guarantee (A) more true — or does it have nothing to do with it?

FRAMEWORK · RECAP

MVP ≠ Smallest App. MVP = Cheapest Test of the Riskiest Assumption.

Swiggy Instamart didn't launch nationally on day one. It climbed a ladder — and every rung was a different kind of MVP, testing a different risk.

1 · Pilot MVP

Launched in a handful of Bengaluru micro-markets before any city-wide rollout — testing whether 10-15 minute delivery was operationally possible at all.

2 · Concierge elements

Early dark-store coordination leaned on manual ops to prove the demand and workflow before automating anything.

3 · Scalability MVP

Only once demand was proven did replenishment and routing get automated — the expensive part came last, not first.

Rule of thumb: if your MVP conversation starts with "what can we cut," you're doing it wrong. It should start with "what's the riskiest thing we don't actually know yet."

FRAMEWORK · CHOOSING A FRAMEWORK

The Framework for the Framework

RICE, MoSCoW, Value-vs-Effort, and Weighted Scoring aren't interchangeable. What fits depends on the shape of the decision in front of you.

	High	Low
Project Complexity	RICE, Weighted Average	MoSCoW, Effort vs Impact
Resource Availability	RICE, Weighted Average	MoSCoW, Effort vs Impact
Stakeholder Involvement	MoSCoW, Weighted Average	RICE, Effort vs Impact
Strategic Alignment	RICE, MoSCoW	Effort vs Impact, Weighted Average
Iterative	RICE, Weighted Average	MoSCoW, Effort vs Impact

FRAMEWORK · MOSCOW

MoSCoW: What's In vs. Out of This Release

MoSCoW doesn't rank a backlog — it decides scope for one fixed release, in four buckets:

Must

The release fails without it.

Should

Important, painful to skip, not fatal.

Could

Nice-to-have, cut first if time runs short.

Won't

Explicitly out of scope this time — so nobody keeps re-asking.

FRAMEWORK · MOSCOW WORKED EXAMPLE

MoSCoW: Airbnb's Very First Launch

Before any of the RICE-scored features on the next slides existed, Airbnb had to ship *something*: renting air mattresses in the founders' own apartment during a sold-out design conference in San Francisco, because every hotel in the city was booked.

Must have

- A simple webpage listing the space (air mattresses + shared bathroom) with photos and a price
- A way for guests to actually contact the founders and confirm a booking — literally email/phone at first

Should have

- Breakfast included in the stay — this became part of the "AirBed & Breakfast" pitch itself
- Basic description of the neighborhood, since guests were flying in blind for a conference

Could have

- Any kind of host profile or bio — guests trusted word of mouth from the conference instead
- Multiple listing options — there was only ever the one apartment

Won't have

- Payments infrastructure — guests paid cash/in person
- Reviews, ratings, or any trust & safety tooling
- Search or browsing — there was nothing to search, only one listing

Teaching point: the "Must have" list here is almost embarrassingly short — and that's correct, not a failure of ambition. Everything in "Won't have" (payments, reviews, trust tooling) is exactly what shows up later as RICE-scored backlog items once there's something worth comparing.

FRAMEWORK · RICE

RICE

RICE = (Reach × Impact × Confidence) ÷ Effort

Reach

How many users/orders this touches per quarter.

Impact

Per-user effect, scored 0.25 / 0.5 / 1 / 2 / 3.

Confidence

How sure you are, as a %.

Effort

Person-months to build.

RICE turns a gut feeling into a number you can put in front of a stakeholder — but the number is only as honest as your Confidence estimate.

FRAMEWORK · RICE WORKED EXAMPLE

Scoring the Instamart Backlog

Same six items you might see on a real Instamart-style backlog, scored and sorted.

Priority	Initiative	Reach	Impact	Confidence	Effort	RICE
1	**Fix inventory-mismatch bug**	**4**	**4**	**90%**	**2**	**7.2**
2	10-min delivery guarantee banner	4	2	40%	1	3.2
3	Dark-store picker route optimization	2	4	70%	3	1.9
4	Smart reorder suggestions	3	3	60%	3	1.8
5	Delivery-partner tracking polish	3	2	50%	2	1.5
6	Personalized recommendations	4	2	30%	4	0.6

Notice: the "quiet win" no one was shouting about — the bug fix — tops the list. The loud CEO ask ranks second. This is exactly why RICE is worth doing even when you're confident you already know the answer.

FRAMEWORK · RICE WORKED EXAMPLE

RICE, Same Product: Airbnb's 0-to-1 Journey

Early Airbnb ("AirBed & Breakfast") had a long list of things it could build next. RICE helps decide what to build first when there's almost no engineering capacity and every feature is competing against the others.

Feature	Reach	Impact	Confidence	Effort	RICE
Professional photography for listings (founders borrowed a camera and shot hosts' apartments themselves)	**4**	**3**	**90%**	**1**	**10.8**
Online payment processing (replacing manual PayPal/cash between host and guest)	5	2	70%	3	2.3
Two-way review system (host reviews guest, guest reviews host)	4	2	50%	2	2.0
Host Guarantee ($50K property-damage protection)	2	3	60%	4	0.9

Notice: the photography experiment wins by a landslide — not because it was technically impressive, but because it was cheap, high-confidence, and touched every listing. This is the real story: Airbnb's founders manually re-photographed listings because they suspected bad photos were killing bookings — and it worked well enough to become a company-wide program. Confidence here isn't a guess about the future — it's "we already watched this work." That's a stronger kind of confidence than a good feeling.

FRAMEWORK · ICE VS RICE

When RICE Is Overkill: ICE

ICE = Impact × Confidence × Effort. No Reach.

Faster, rougher, and useful when you genuinely don't trust your reach numbers yet — very early stage, or a brand-new product area with no usage data. The trade-off: without Reach, ICE can't tell a niche-but-intense feature apart from a broad-but-shallow one. Use the Framework-for-Framework table from Slide 6 to decide which one earns its complexity for the decision in front of you.

FRAMEWORK · KANO MODEL

The Kano Model: Not All Impact Is the Same Kind

RICE and ICE tell you *how much* impact. Kano tells you *what kind.*

Basic

Invisible when present, painful when absent. Table stakes.

Performance

Satisfaction scales linearly with how well you do it.

Delighter

Unexpected. Nobody asked for it, but it differentiates you.

Now classify three real Instamart-style features:

UPI payment success (no failed transactions)

- Basic
- Performance
- Delighter

0 responses so far

- **Your response only (live sync unavailable)**

10-minute delivery speed

- Basic
- Performance
- Delighter

0 responses so far

- **Your response only (live sync unavailable)**

Playful order-tracking animation / rewards

- Basic
- Performance
- Delighter

0 responses so far

- **Your response only (live sync unavailable)**

FRAMEWORK · KANO REVEAL

Kano Reveal

Basic

UPI payment success

Nobody thanks you for it working. Everybody leaves a 1-star review when it doesn't. RICE will underrate this because Reach looks "normal" — Kano is what tells you it's non-negotiable.

Performance

10-min delivery speed

8 minutes beats 10, which beats 12. Every improvement is felt proportionally — this is where competitive pressure lives.

Delighter

Tracking animation / rewards

High delight-per-rupee early on, but delighters decay into "basic" expectations over time as competitors copy them.

ROADMAPPING

Now-Next-Later: A Communication Tool, Not an Execution Tool

Sprint plans are for engineers. Now-Next-Later is for stakeholders — it buys you the right to say "not now" without saying "never."

NOW

Inventory-mismatch fix
Dark-store picker route optimization

NEXT

Smart reorder suggestions
Delivery-partner tracking polish

LATER

Personalized recommendations
10-min guarantee banner (revisit once ops is fixed)

FRAMEWORK · COST OF DELAY

Cost of Delay and CD3

CD3 = Cost of Delay ÷ Job Size. It's the variable RICE completely ignores: *time*.

Worked example

A dark-store safety-compliance audit feature scores a modest 1.2 on RICE — low reach, moderate effort. But there's a regulatory deadline in 6 weeks. Before the deadline, delay costs little. After it, the cost of delay approaches infinite (fines, forced shutdowns). CD3 flips this item from "somewhere in the middle" to "#1, non-negotiable" — even though nothing about its RICE score changed.

Takeaway: a framework that ignores time will always underrate deadlines. That's not a flaw to fix by adding more variables — it's a signal to switch lenses.

ACTIVITY 2 · SHORT (8 MIN)

Defend the Call

Pairs. One plays PM, one plays a VP of Sales who wants "Personalized recommendations" (RICE: 0.6, dead last) prioritized — "a client asked for it personally." 60 seconds each way, then rotate.

8:00 Start Reset

Debrief poll: after both rounds —

Which phrase de-escalates without over-promising?

- **"I'll put it on the roadmap and get back to you."**
- **"Here's what it's competing against, and why — what would you deprioritize instead?"**
- **"That's not really feasible right now."**

0 responses so far

- **Your response only (live sync unavailable)**

CASE STUDY A · FIGURE IT OUT

Swiggy Instamart: 10 Features, One Backlog

Dark-store operational tooling competes for the same roadmap slots as consumer-facing features. Don't sort these into "ops" and "consumer" buckets yet — some aren't what they look like.

#	Feature
1	Real-time inventory accuracy sync between dark stores and the app
2	Picker route optimization inside dark stores
3	Personalized product recommendations on the homepage
4	One-tap reorder of a previous basket
5	Dark-store staff scheduling and shift-management tool
6	Live order-tracking map for customers
7	Automated demand forecasting for restocking
8	In-app loyalty and rewards points program
9	Cold-chain temperature monitoring for perishable items
10	Voice search for product discovery

In your groups, figure out:

- Which of these are genuine Trade-offs (one side clearly loses) — and which are actually Win-Win (Strategic) in disguise, where an "ops" fix is secretly a consumer feature? (Revisit Slide 4.)
- Pick any 3 and RICE-score them. Which one surprised you most once you had a number?
- If you were the ops lead losing this trade-off every quarter, what would you ask the PM to change about how RICE gets scored?

CASE STUDY B · FIGURE IT OUT

Notion: 10 Features, One Backlog

Platform/API investment compounds slowly and is hard to estimate; consumer feature requests are immediate, visible, and easy to score. Same instructions as Instamart — don't pre-sort them.

#	Feature
1	Public API v2 with webhook support
2	Native mobile widget for quick note capture
3	Third-party app marketplace / integrations directory
4	AI-powered "ask your workspace" search
5	Granular permissions system for enterprise teams
6	Dark mode customization options
7	Bulk data export/import SDK for partners
8	Real-time collaborative cursors on a page
9	Single sign-on (SSO) and SCIM provisioning
10	Community-curated templates gallery

In your groups, figure out:

- Which of these are platform bets vs. consumer feature requests — and which would score lowest on RICE simply because Reach and Confidence are hard to estimate, even though they might matter most in three years?
- Pick any 3 and estimate Cost of Delay instead of RICE for them. Does the ranking change?
- What would you have to change about how you score platform work to give it a fair fight against consumer features?

LONG ACTIVITY · BRIEFING

Live Backlog Prioritization — In Jira / Azure DevOps

This is the part a language model cannot do for you: real-time judgment under social pressure, executed inside the actual tool you'll use on the job.

Round 1 — Baseline (10 min)

Score the top 20 items of a 100-item anonymized Instamart-style backlog using RICE, entering values directly into the tool's custom fields. Sort the backlog by RICE.

Round 2 — Constraints (15 min)

New constraints will be injected live. You must re-rank the backlog inside the tool in real time — not on paper, not in a spreadsheet.

Round 3 — Defend (10 min)

Present your final top-10 to the room. You must defend at least two calls that look counterintuitive on paper.

35:00 Start Reset

LONG ACTIVITY · CONSTRAINT INJECTION

Constraint Cards

Facilitator: reveal these one at a time during Round 2. Each one forces a live re-rank inside Jira/Azure DevOps — visible, undoable, discussed.

01

Legal flags 3 backlog items as compliance-mandatory, with a regulatory deadline in 6 weeks. Cost of Delay just went to near-infinite on these three — regardless of their RICE score.

02

Engineering capacity is cut 30% for next sprint. Re-rank knowing every Effort estimate on the board just got more expensive.

03

Your biggest enterprise dark-store partner threatens to leave over item #47 — which scored near the bottom of your original RICE sort.

LONG ACTIVITY · DEBRIEF

Debrief

- Which constraint forced you to override the framework outright?
- Did anyone actually re-score Effort after the capacity cut — or did the group just accept shipping less without adjusting the numbers?
- How did you explain the override to your teammates in the tool — a comment, a re-rank with no explanation, something else?

Which constraint forced the biggest override of your original ranking?

Compliance deadline (Card 01)

Capacity cut (Card 02)

Partner threat (Card 03)

0 responses so far

Your response only (live sync unavailable)

SYNTHESIS

A Framework Is a Hypothesis, Not a Verdict

> **Your job isn't to compute the hypothesis. It's to know when to trust it, when to override it — and how to say so out loud without losing the room.**

Let's go back to where we started.

Same scenario as Slide 2 — which do you ship this sprint?

A — Ship the 10-minute guarantee banner

B — Fix the inventory-mismatch bug

C — Ship smart reorder suggestions

0 responses so far

- **Your response only (live sync unavailable)**

Same scenario, same options. If your answer changed — good. That's the framework doing its job.

TAKE-HOME CHECKLIST

Before You Get Down to Your MVP

Ten questions to answer before you build anything — check them off as you go.

- [ ] Exactly what problem are you trying to solve? (Value proposition)
- [ ] For whom do we solve that problem? (Target market)
- [ ] How big is the opportunity? (Market size)
- [ ] How will you measure success? (Metrics/revenue strategy)
- [ ] What alternatives are out there? (Competitive landscape)
- [ ] Why are we best suited to pursue this? (Our differentiator)
- [ ] Why is now the best time to introduce this product? (Market window)
- [ ] How will you bring this product to market? (Go-to-market strategy)
- [ ] What factors are critical to success? (Solution requirements)
- [ ] Given the above, what's the recommendation? (Build an MVP or not?)